

TP7 Java Yassine Hamrouni

Exercice 1 - Arbre d'expressions

1- On va créer Expr dans un fichier Expr.java

```
package fr.uge.calc;

public sealed interface Expr permits Value, Add, Sub, Mul{
    public int eval();
}
```

2- On souhaite maintenant pouvoir évaluer (trouver la valeur) d'une expression (Expr) en appelant la méthode eval comme ceci

```
package fr.uge.calc;

public record Value(int value) implements Expr {
    @Override
    public int eval(){
        return value;
    }
}

// fr/uge/calc/Add.java
package fr.uge.calc;

public record Add(Expr left, Expr right) implements Expr {
    @Override
    public int eval() {
        return left.eval() + right.eval();
    }
}

package fr.uge.calc;

public record Sub(Expr left, Expr right) implements Expr {
    @Override
    public int eval() {
        return left.eval() - right.eval();
    }
}

// fr/uge/calc/Mul.java
```

```

package fr.uge.calc;

public record Mul(Expr left, Expr right) implements Expr {
    @Override
    public int eval() {
        return left.eval() * right.eval();
    }
}

```

3 et 6 Écrire une méthode parse qui prend un Scanner en entrée et crée l'arbre d'expression. Noter que prendre un Scanner en paramètre ne permet pas de ré-utiliser la méthode parse si, par exemple, l'expression à parser est stockée dans une List de String.

Quelle interface que doit-on utiliser à la place de Scanner pour que l'on puisse appeler la méthode parse avec un Scanner ou à partir d'une List.

L'interface utilisait sera un Iterator de String.

```

public class Parser {
    public static Expr parse(Iterator<String> tokens) {
        if (!tokens.hasNext()) {
            throw new IllegalArgumentException("Expression incomplete or malformed");
        }

        String token = tokens.next();

        switch (token) {
            case "+" -> new Add(parse(tokens), parse(tokens));
            case "-" -> new Sub(parse(tokens), parse(tokens));
            case "*" -> new Mul(parse(tokens), parse(tokens));
            default -> {
                try {
                    int value = Integer.parseInt(token);
                    return new Value(value);
                } catch (NumberFormatException e) {
                    throw new IllegalArgumentException("Invalid token: " + token);
                }
            }
        };
        throw new IllegalArgumentException("Invalid token: " + token);
    }
}

```

4- Il y a un bug dans le code que l'on a écrit, on permet à n'importe qui d'implanter Expr mais cela ne marchera pas avec la méthode parse qui elle liste tous les sous-types possibles.

###Comment corriger ce problème ?

Il faut utiliser une interface scelle

```
package fr.uge.calc;
```

```
public sealed interface Expr permits Value, BinOp {  
    int eval();  
}
```

5 Déplacer le main dans une nouvelle classe Main dans le package fr.uge.calc.main et faire les changements nécessaires.

```
package fr.uge.calc.main;
```

```
import fr.uge.calc.Add;  
import fr.uge.calc.Expr;  
import fr.uge.calc.Sub;  
import fr.uge.calc.Value;
```

```
public class Main {  
    public static void main(String[] args) {  
        //      Expr expression = new Add(new Value(2), new Value(3));  
        //      Expr expression2 = new Sub(new Mul(new Value(2), new Value(3)), new Value(4));  
  
        Expr expression = new Add(new Value(2), new Value(3));  
        Expr expression2 = new Sub(new Add(new Value(2), new Value(3)), new Value(4));  
        System.out.println(expression2.eval());  
    }  
}
```

7 Écrire la méthode d'affichage de l'arbre d'expression pour que l'affichage se fasse dans l'ordre de lecture habituel.

Note : il va falloir ajouter des parenthèses et peut-être des parenthèses inutiles !

```
Add:  
@Override
```

```
String ToString() {
    var sb = new StringBuilder();
    return sb.append("(").append(left()).append(" ").append("+").append(" ").append(right()).append(")");
}
```

Sub:

```
@Override
String ToString() {
    var sb = new StringBuilder();
    return sb.append("(").append(left()).append(" ").append("-").append(" ").append(right()).append(")");
}
```

Mul:

```
@Override
String ToString() {
    var sb = new StringBuilder();
    return sb.append("(").append(left()).append(" ").append("*").append(" ").append(right()).append(")");
}
```

8 Enfin, on peut voir que le code de eval dans Add, Sub et Mul est quasiment identique, dans les trois cas : la méthode eval est appelée sur left et right. On souhaite factoriser ce code (on ne le ferait probablement pas dans la vraie vie car il n'y a pas assez de code à partager, mais ce n'est pas la vraie vie, c'est un exercice) en introduisant un type intermédiaire BinOp, sous-type de Expr et super-type de Add, Sub et Mul.

Le type BinOp doit-il être un record, une classe ou une interface ?

Il faut utiliser une interface scelle qui herite de Expr

```
package fr.uge.calc;

public sealed interface BinOp extends Expr permits Add, Sub, Mul {
    Expr left();

    Expr right();

    int applyOp(int left, int right);

    @Override
    default int eval() {
```

```

        return applyOp(left().eval(), right().eval());
    }

    default String operationToString(String operator) {
        var sb = new StringBuilder();
        return sb.append("(").append(left()).append(" ").append(operator).append(" ").append(right())
    }
}

```

9 Sachant que l'on veut écrire eval dans BinOp, comment eval doit être déclarée ? Et comment, dans eval de BinOp, peut-on accéder aux champs left et right, qui sont déclarés dans Add, Sub et Mul ?

Pour cela il faut les déclarer dans l'interface Binop:

```

package fr.uge.calc;

public sealed interface BinOp extends Expr permits Add, Sub, Mul {
    Expr left();

    Expr right();
}

```

10 Écrire le code de BinOp (dans BinOp.java) et modifier Add, Sub et Mul en conséquence.

```

public sealed interface BinOp extends Expr permits Add, Sub, Mul {
    Expr left();

    Expr right();

    int applyOp(int left, int right);

    @Override
    default int eval() {
        return applyOp(left().eval(), right().eval());
    }

    default String operationToString(String operator) {
        var sb = new StringBuilder();
        return sb.append("(").append(left()).append(" ").append(operator).append(" ").append(right())
    }
}

```

```

    }

}

public record Add(Expr left, Expr right) implements BinOp {
    @Override
    public Expr left() {
        return left;
    }

    @Override
    public Expr right() {
        return right;
    }

    @Override
    public int applyOp(int left, int right) {
        return left + right;
    }

    @Override
    public String toString() {
        return operationToString("+");
    }
}

package fr.uge.calc;

public record Sub(Expr left, Expr right) implements BinOp {
    @Override
    public Expr left() {
        return left;
    }

    @Override
    public Expr right() {
        return right;
    }

    @Override
    public int applyOp(int left, int right) {
        return left - right;
    }

    @Override
    public String toString() {

```

```

        return operationToString("-");
    }
}

package fr.uge.calc;

public record Mul(Expr left, Expr right) implements BinOp {
    @Override
    public Expr left() {
        return left;
    }

    @Override
    public Expr right() {
        return right;
    }

    @Override
    public int applyOp(int left, int right) {
        return left * right;
    }

    @Override
    public String toString() {
        return operationToString("*");
    }
}

```